

2025년 게임프로그래밍전문가 실기시험

[유의사항]

1. 문제지는 지급된 USB에 HWP와 PDF 형태로 있음
2. 각 문제 폴더에는 답안 제작에 활용될 자료가 포함되어 있음
 - 문제(답안작성용) 폴더의 VS2022.zip이 시험문제용 프로젝트 파일임
 - VS2022.zip에는 Visual Studio 2022용 프로젝트가 포함되어 있음
 - Visual Studio 2022 이외의 프로젝트는 제공되지 않으며, 이 버전 이외의 개발도구를 사용하는 경우 수험생이 개별적으로 변경해야함. 이로 인해 발생한 모든 문제는 수험자에게 귀책됨
 - 실행파일(참고용) 폴더는 수험시 참고하기 위한 실행파일이 포함되어 있음
- ※ 제공되는 기본 제공 코드는 수험자 임의 변경 불가하며, 이로 인해 야기된 모든 문제와 불이익은 수험자에게 귀책됨을 알려드립니다.
3. Visual Studio의 Build Target은 필요시 OS(32/64bit)에 맞춰서 직접 설정해야 함
4. 답안은 각 문제 당 다음 사항을 제출해야 함
 - 컴파일 가능한 완전한 프로젝트 소스
 - 컴파일된 실행파일 (파일명 : [문제번호]수험번호_이름.exe)
예) [문제4]130000112233_홍길동.exe
 - 각 문제에서 명시적으로 요구하는 결과물
5. 제작된 개별 답안은 하나의 폴더에 넣은 후 폴더명 변경(수험번호_이름)
 - 압축하여 압축파일도 제출(제출한 답안의 파손 및 임의삭제 방지 목적)
6. 채점 기준
 - 제출된 프로젝트 소스파일을 컴파일하여 실행 상태를 확인할 수 있어야 함
 - 실행 결과와 소스코드를 모두 평가함.
 - 실행화면영역(Client Area) 좌측 상단에 수험번호와 이름이 출력되어야 함.
수험번호와 이름 출력 누락시 0점 처리할 수 있음.
7. 제출확인서 서명 필

게임국가기술자격검정본부

문화체육관광부 한국콘텐츠진흥원

[공통 사항] 실기 시험문제 2025

[제공 자료]

- ◎ 답안 작성에 필요한 프로젝트 소스 (폴더명 - 문제(답안작성용))
- ◎ 실행파일 형태의 참고 프로그램 (폴더명 - 실행파일(참고용))

[제출 답안]

- ◎ 컴파일 가능한 완전한 프로젝트 소스파일
- ◎ 실행파일
- ◎ 실행화면영역(Client Area) 좌측 상단에 수험번호와 이름이 출력되어야 함.
수험번호와 이름 출력 누락시 0점 처리할 수 있음.

[채점기준]

- ◎ 제출된 프로젝트 소스파일을 컴파일하여 완전한 실행 상태를 확인할 수 있어야 함
- ◎ 실행결과와 소스코드를 모두 평가함.
- ◎ 실행화면영역(Client Area) 좌측 상단에 수험번호와 이름이 출력되어야 함.
수험번호와 이름 출력 누락시 0점 처리할 수 있음.

문제 1. C++ 언어로 유한 상태 기계(Finite State Machine, 이하 FSM)를 활용하여 게임 상태를 관리하는 프로그램을 작성하였다. 아래 설명과 요구사항을 참고하여 프로그램이 정상적으로 동작하도록 FSM.cpp의 빈칸을 채우시오.

[배점 : 30점]

[문제 설명]

- ◎ 게임 오브젝트의 상태 관리로 많이 사용되는 유한 상태 기계(Finite State Machine)을 프로그램으로 제작했다.
- ◎ 상태는 Idle, Move, Attack 세가지 상태가 존재한다. FSM.h 및 FSM.cpp 파일에 작성된 IdleState, MoveState, AttackState 클래스는 부모클래스 State를 상속받아 각각의 상태 관련 처리를 한다.
- ◎ FSM.h 및 FSM.cpp 파일에 작성된 Fsm 클래스는 유한 상태 기계를 구현한 클래스이며 주요 함수로는 아래의 함수들이 있다.
 - CreateState : 상태 클래스의 인스턴스를 생성하는 함수
 - Transition : 상태 전환을 담당하는 함수
 - Update : 일정 시간 마다 유한 상태 기계를 갱신하는 함수
 - 입력을 외부에서 전달받는 함수들(SetInputs, GetInputs)
- ◎ 상태 전환 관련 규칙은 아래와 같다.
 - Move 상태는 Idle 상태일 때 W, A, S, D 키입력을 통해 전환된다.
 - Attack 상태는 Idle 혹은 Move 상태일 때 Space 키입력을 통해 전환된다. 일단 Attack 상태로 전환되면 키입력으로는 다른 상태로 전환되지 않으며, 0.5초가 지난 후 Idle 상태로 전환된다.
 - Idle 상태는 최초로 기본으로 설정되며, Move 상태가 끝나거나 Attack 상태가 끝나면 다시 Idle 상태로 전환된다.
- ◎ Main.cpp 에는 main 함수가 작성되어 있고, 100ms의 주기로 Fsm 클래스의 인스턴스를 갱신한다.
- ◎ **문항 1** : FSM.cpp 파일에는 Fsm::CreateState 함수가 있으며, 이 함수 안에 있는 문항 1을 완성하십시오. (배점 10점)

```
State* Fsm::CreateState(STATE_TYPE state)
{
    //-----
    //★ 문항 1 : nullptr 대신 전달받은 stateType 에 맞게 State 클래스의 인스턴스를 생성하여 반환하십시오.
    //          STATE_IDLE 을 전달 받았을 경우 IdleState 클래스의 인스턴스 생성후 반환
    //          STATE_MOVE 을 전달 받았을 경우 MoveState 클래스의 인스턴스 생성후 반환
    //          STATE_ATTACK 을 전달 받았을 경우 AttackState 클래스의 인스턴스 생성후 반환
    //-----
    return nullptr;
}
```

- ◎ **문항 2** : FSM.cpp 파일에는 MoveState::Update 함수가 있으며, 이 함수 안에 있는 문항 2를 완성하

시오. (배점 10점)

```
void MoveState::Update()
{
    //-----
    //★ 문항 2 : 키 입력이 없을 경우 IDLE 상태로 변경하고
    //             INPUT_ATTACK 키가 입력되었을 경우 ATTACK 상태로 변경하시오.
    //             상태 변경이 되지 않을경우 아래의 콘솔 출력 부분이 실행되도록 하시오.
    //-----

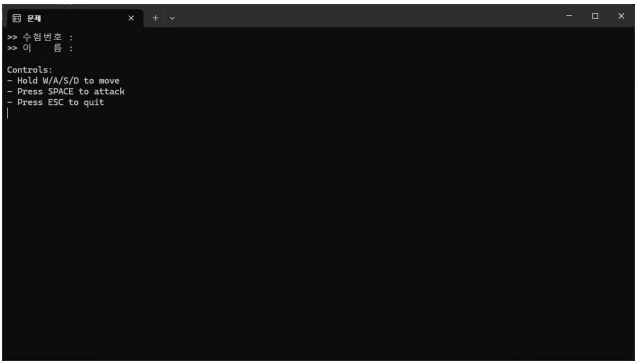
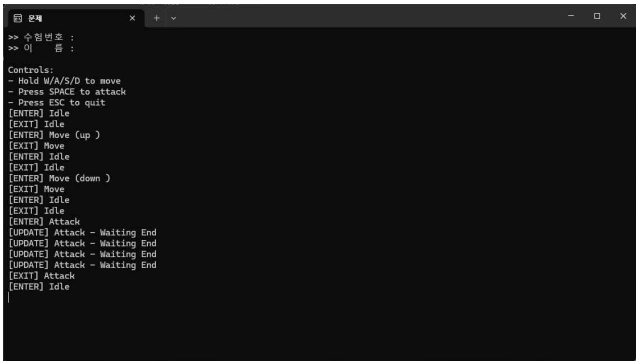
    //상태 변경이 되지 않을경우 실행
    _direction = CalculateDirection();
    std::cout << "[UPDATE] Move - " << _direction << std::endl;
}
```

◎ 문항 3 : FSM.cpp 파일에는 AttackState::Update 함수가 있으며, 이 함수 안에 있는 문항 3을 완성 하시오. (배점 10점)

```
void AttackState::Update()
{
    //-----
    //★ 문항 3 : 키입력은 무시하고 ATTACK 상태가 시작된지 0.5초를 초과하면 IDLE 상태로 전환하시오.
    //             상태가 시작된 시간은 _start 변수에 저장되어 있음
    //             상태 변경이 되지 않을경우 아래의 콘솔 출력 부분이 실행되도록 하시오.
    //-----

    //상태 변경이 되지 않을경우 실행
    std::cout << "[UPDATE] Attack - Waiting End" << std::endl;
}
```

◎ 수정 없이 프로그램을 실행하면 키입력을 해도 아무런 반응 없이 [그림1]과 같이 출력되며, 문제를 해결하고 실행 후 키입력을 하면 [그림2]와 같은 화면이 출력된다. 02_실행파일(참고용) 폴더에 있는 Exam.exe 를 실행하면 [그림2]화면을 확인할 수 있다.

	
[그림1] 수정없이 실행할 경우 화면	[그림2] 문제를 해결한 후 실행 화면

[제공 자료]

◎ 02_실행파일(참고용) 폴더

- Exam.exe : 참고용 정답 클라이언트 실행파일

◎ 01_문제(답안작성용) 폴더

- 응시자가 작성해야 하는 클라이언트 프로그램
- 프로젝트-소스의 ‘★’ 부분을 확인, 참고할 것.
- 작성 환경 및 라이브러리 : VisualStudio 2022 / Win32 API
- ※ 코드내 수험번호와 이름을 기입하는 부분이 있으므로 꼭 클라이언트 실행시 출력되도록 할 것.

문제 2. 수식 트리 기반의 계산기를 구현하고자 한다. 다음의 [문제 설명]과 [상세 조건 설명]을 이용하여 요구하는 사항을 구현하라. 단, 모든 문항에서 요구하는 기능들은 반드시 모범 답안과 일치해야 한다. [배점 : 총 30점]

[문제 설명]

- ◎ 수식 트리는 이진 트리의 한 형태로, 리프 노드에는 피연산자(숫자), 내부 노드에는 연산자(+, -, ×, ÷)가 저장된다.
본 문제에서는
 - (1) 중위 표기식을 후위 표기식으로 변환하고,
 - (2) 후위 표기식을 이용해 수식 트리를 생성한 뒤,
 - (3) 재귀적으로 트리를 후위 순회하면서 결과를 계산하는 수식 계산기를 완성한다.
- ◎ 구현 대상이 되는 기본 자료구조(노드 정의, 좌/우 자식 연결, 우선순위 함수, 중위→후위 변환, 트리 평가)의 인터페이스는 문제에서 제공하는 프로젝트를 따른다.

[상세 조건 설명]

- ◎ 소스 코드 exp_tree_02.cpp 내의 실행화면 타이틀 상에 수험번호와 성명이 출력되도록 수정하십시오.
(수정하지 않을 경우 감점 있음)
- > int main() 함수 내에서

```
//
// ★수험번호, 성명 작성 >> 반드시 수정 필수(수정 안하면 감점)
//
system("cls");
// 아래의 수험번호와 성명을 본인 정보로 수정하세요.
system("title KOCCA:[문제2](답안작성용), 수험번호:nnnnnnnn, 성명:홍길동");
system("mode con:lines=12 cols=45");
cout << "=====Wn"
cout << " KOCCA:[문제2](답안작성용)Wn"
cout << " 수험번호: nnnnnnnnWn"
cout << " 성명: 홍길동Wn"
cout << "=====WnWn"
//
// ★수험번호, 성명 작성(수정 안하면 감점)
//
```

- ◎ 위의 수식 트리에 대한 문제 설명을 충분히 인지하고 소스 코드 exp_tree_02.cpp 내의 아래 부분에 해당하는 주석 부분을 [그림2] 최종화면 예시에 맞도록 코드를 작성하여 완성한다.

[문제 (A)] 10점

◎ 아래 설명을 보고 비어있는 부분에 알맞은 코드를 작성하세요.

```
// 연산자 우선순위 함수, op는 연산자 문자('+', '-', '*', '/')
int GetPriority(char op) {
    //
    // /★문제( A )
    //
    // *, / : 5 (가장 높은 우선순위)
    // +, - : 3 (중간 우선순위)
    // 우선순위에 맞도록 연산자와 반환값을 수정하십시오.
    if (op == '+' || op == '-') return 3;
    if (op == '*' || op == '/') return 5;
    //
    // /★문제( A )
    //
    // ( : 1 (괄호는 가장 낮은 우선순위로 처리)
    if (op == '(') return 1;
    // 나머지 문자(예: 공백 등) : -1 (우선순위 없음)
    return -1;
}
```

[문제 (B)] 10점

◎ 아래 함수 내의 문제 부분의 설명을 보고 알맞은 코드를 작성하세요.

```
// 중위표기법-> 후위표기법 변환(토큰(숫자, 연산자등) 단위, 숫자는 최대10자리)
void InfixToPostfix(const char* inStr, std::vector<std::string>& outTokens) {
    ...
    //
    // ★문제( B )
    //
    // 연산자를 만났을때, 스택에 있는 연산자들과 우선순위를 비교하여
    // 현재 연산자(token)의 우선순위가 낮거나 같으면 스택의 연산자를 꺼내서
    // (outTokens에 추가)
    // 후위표기법으로 변환한다. 이후 현재연산자를 스택에 push한다.
    case '+': case '-': case '*': case '/':
        // 1. 스택이 비어있지 않고, 현재 연산자(token)의 우선순위가
        // 스택의 top에 있는 연산자보다 낮거나 같을 때 반복
        while (!strStack.empty() && GetPriority(token) <= GetPriority(strStack.top())) {
            // 2. 스택 top에 있는 연산자(문자 1개)를 후위표기법 결과에 추가
            // 스택 top 연산자를 문자열로 변환하여 추가하도록 수정하십시오.
            outTokens.push_back(strStack.top());
            strStack.pop();
            // 3. 스택에서 top 연산자를 제거(pop)하도록 아래 내용을 수정하십시오.
        }
        strStack.push(token);
    }
}
```

```

    }
    // 4. 현재 연산자를 스택에 push
    strStack.push(token);
    break
//
// ★문제( B )
//
...
}

```

[문제 (C)] 10점

◎ 아래 설명을 보고 비어있는 부분에 알맞은 코드를 작성하세요.

```

// 후위표기법(예: "2 3 + 4 *") 토큰을 받아서, 이진 수식 트리를 생성
// tokens: 후위표기법으로 변환된 토큰(숫자, 연산자등) 리스트
// 반환값: 수식 트리의 루트 노드 포인터
BTreeNode* MakeExpTree(const std::vector<std::string>& tokens) {
    ...
    //
    // ★문제( C )
    //
    // 2. 연산자라면
    // 스택에서 오른쪽 자식 노드 꺼내 연결
    MakeRightSubTree(pnode, stack.top());
    // 스택에서 제거하는 코드를 추가하세요.
    stack;
    // 스택에서 왼쪽 자식 노드 꺼내 연결
    MakeLeftSubTree(pnode, stack.top());
    // 스택에서 제거하는 코드를 추가하세요.
    stack;
    // 노드에 연산자 데이터 저장하는 함수를 추가하세요.
    SetData;
    //
    // ★문제( C )
    //
}
// 3. 현재 노드를 스택에 push
stack.push(pnode);
}
...
}

```

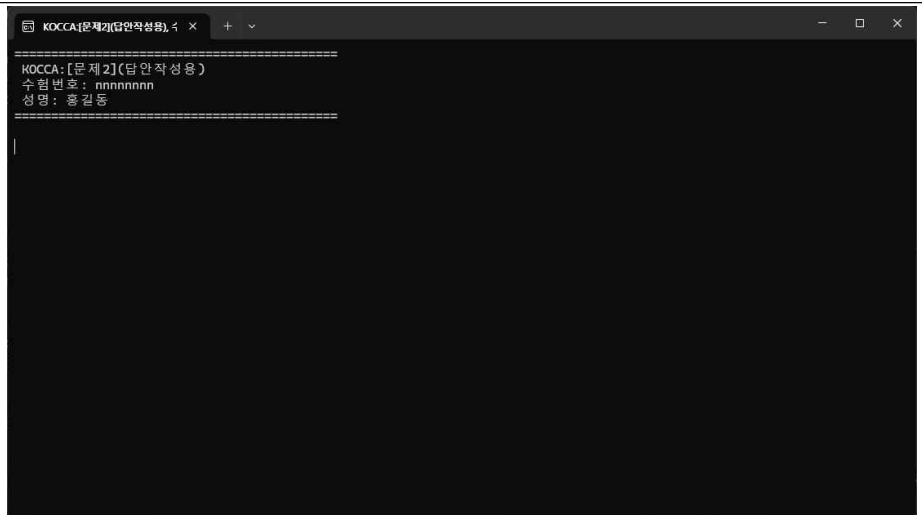
[제공 자료]

◎ 답안 작성에 필요한 소스

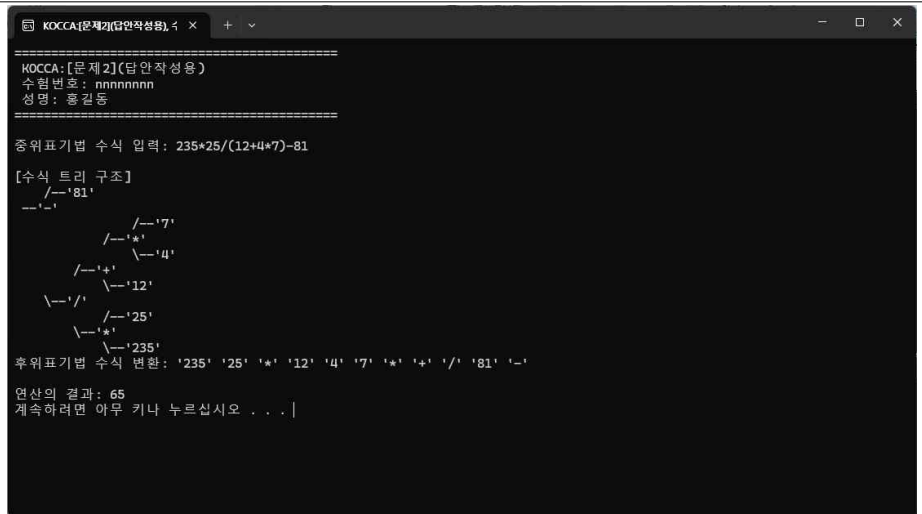
◎ 실행파일 형태의 참고 프로그램

- “답안 작성용” 프로젝트가 제공됨.
- “답안 작성용” 프로젝트 소스 코드의 ‘// 문제’ 부분을 확인, 참고할 것.
- 작성 환경 및 라이브러리 : VisualStudio 2022

[그림1] 문제화면 예시



[그림2] 최종화면 예시



문제 3. 사용자가 입력한 크기의 오목 게임을 구현하세요. 두 명의 플레이어가 번갈아가며 돌을 놓고, 가로, 세로, 대각선 중 어느 방향으로든 5개의 돌을 연속으로 놓으면 승리하는 게임입니다.
[배점 : 20점]

[게임 규칙]

- 게임 시작 시 사용자가 보드 크기를 입력 (최소 5, 최대 20)
- 흑돌(X)이 먼저 시작
- 가로, 세로, 대각선으로 5개 연속 배치 시 승리
- 이미 돌이 놓인 자리에는 돌을 놓을 수 없음
- 게임 종료는 -1 -1 입력

[출력 예시]

```
수험자: 홍길동 수험번호: 0000
오목 게임에 오신 것을 환영합니다!
게임판 크기를 입력하세요 (권장: 15): 15
=====
오목 게임
=====
흑돌(X)이 먼저 시작합니다.
5개의 돌을 연속으로 놓으면 승리!
게임 종료는 -1 -1을 입력하세요.
=====

  0 1 2 3 4 5 6 7 8 9 10 11 12 13 14
0 . . . . . . . . . . . . . . .
1 . . . . . . . . . . . . . . .
2 . . . . . . . . . . . . . . .
3 . . . . . . . . . . . . . . .
4 . . . . . . . . . . . . . . .
5 . . . . . . . . . . . . . . .
6 . . . . . . . . . . . . . . .
7 . . . . . . . . . . . . . . .
8 . . . . . . . . . . . . . . .
9 . . . . . . . . . . . . . . .
10 . . . . . . . . . . . . . . .
11 . . . . . . . . . . . . . . .
12 . . . . . . . . . . . . . . .
13 . . . . . . . . . . . . . . .
14 . . . . . . . . . . . . . . .

플레이어 흑돌(X) 차례입니다. (수: 1)
행과 열을 입력하세요 (0-14) 또는 종료(-1 -1): |
```

[구현해야할 코드]

자세한 구현 내용은 아래의 구현해야 할 부분을 참고할 것.

main.cpp파일에서 코드의 성명 및 수험번호를 본인의 성명 및 수험번호로 변경할 것
반드시 소스코드를 제출해야함. 미제출시 0점 처리.

■ 구현해야할 코드

(문제 1)

```
bool Omok::makeMove(int row, int col) {
    // 범위 검사 및 빈 자리 확인
    if (row < 0 || row >= boardSize || col < 0 || col >= boardSize ||
        ★ 문제(1) ) {
        return false;
    }

    // 현재 플레이어의 돌을 놓음
    board[row][col] = currentPlayer;
    return true;
}
```

(문제 2)

```
void Omok::switchPlayer() {
    // 흑돌(X)과 백돌(O) 교체
    currentPlayer = (currentPlayer == 'X') ? ★문제(2) : ★문제(2);
}
```

[채점 기준]

항목	배점	평가 내용
문제 (1)	10점	유효성 검사 로직 (이미 돌이 놓인 위치 확인)
문제 (2)	10점	플레이어 교체 로직

총점: 20점

* 문제(2)는 모두 정답인 경우에만 10점 처리, 그렇지 않은 경우에 0점 처리

■ 수정불가 코드

- 구현해야 할 부분에 명시된 것 이외의 모든 구역
- 게임의 Scene이 0.1초당 1회 업데이트 되는 로직
- 타겟의 hp, 크기, 이동 속도

[제공 자료]

◎ 실행파일(참고용)

- OmokGame.exe : 참고용 실행파일(문제를 해결 후 실행하여 확인에 사용)

◎ 문제(답안 작성용) 폴더

- 응시자가 작성해야 하는 프로그램
- 프로젝트-소스의 '★' 부분을 확인, 참고할 것.(이외의 영역에서 소스 코드가 변경된 경우 0점 처리될 수 있음)
- 작성 환경 및 라이브러리 : VisualStudio 2022

문제 4. 다음의 문제에서 제시된 각 기능들을 구현하라. 단, 모든 문항에서 요구하는 기능들은 반드시 모범답안과 일치해야 한다. [배점 : 20점]

Texture UV애니메이션을 할 수 있도록 셰이더 코드를 수정하시오. **반드시 셰이더 소스코드를 제출해야함.**
미제출시 0점 처리.

[문제설명]

- ◎ 아래 [그림] 이미지를 참고하여 텍스처 애니메이션을 구현하시오.
- ◎ 키보드의 좌우 화살표키를 사용하여 물체에 대한 회전이 가능함

[상세 조건 설명]

- ◎ float2 uv = In.T; // ★ texture offset계산으로 uv애니메이션 구현
- 이 코드의 In.T값을 적절하게 조작하여 최종 uv좌표의 Tiling, Offset기능을 구현하시오.

[소스작성 요령]

- ◎ 셰이더 이외의 영역에서 소스 코드가 변경된 경우 0점 처리될 수 있음.

